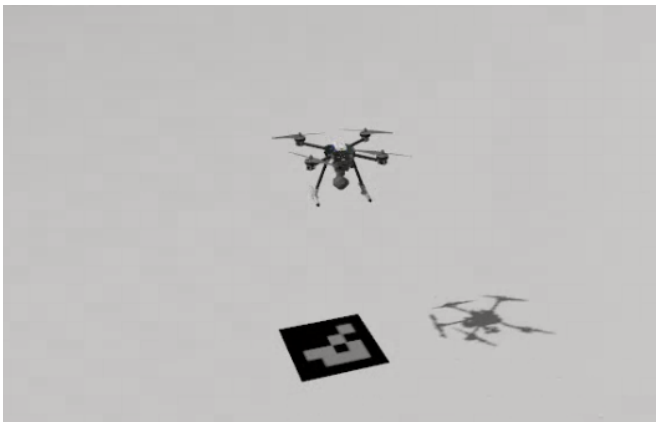
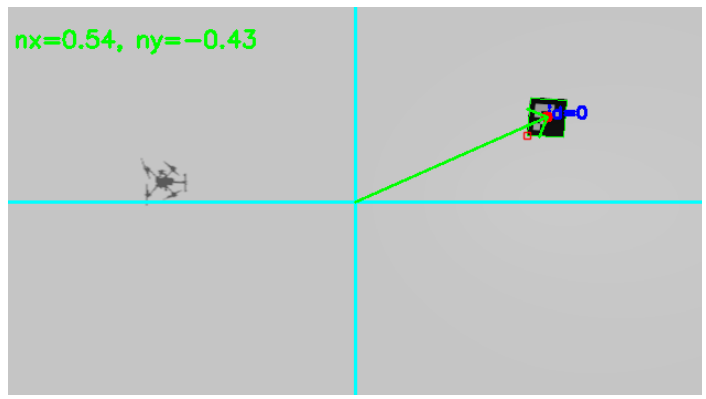
Autonomous Vision-Based Precision Landing System for Multirotors

Complete Beginner's Guide — Documentation & Full Code Walkthrough

Stack: PX4 Autopilot · QGroundControl · Gazebo · ROS 2 · Python / MAVSDK · OpenCV · ArUco



(a) Gazebo simulation: drone approaching the ArUco landing pad.



(b) Camera debug view: crosshair, detected marker, and live error values (`nx=0.02, ny=0.01`).

Figure 1: The precision landing system in simulation. **Left**: third-person Gazebo view of the drone and marker. **Right**: the drone's downward camera feed with ArUco detection overlay. The cyan crosshair marks the image centre; the green arrow shows the error vector the controller must drive to zero before landing.

Abstract — This document is a complete, beginner-friendly technical guide to building an Autonomous Vision-Based Precision Landing system for multirotor drones. No prior experience with robotics, drones, or computer vision is assumed. The system guides a drone from return-to-launch, through ArUco-marker detection and alignment, to a precise touchdown—fully automatically. Every concept is explained from first principles, and every line of both Python scripts is annotated.

1 Introduction

The ability to land a multirotor precisely on a designated target is a core requirement for package delivery, autonomous recharging, and swarm coordination. This project implements such a system in software-in-the-loop (SITL) simulation using entirely open-source tools.

Goal. The drone autonomously locates a printed ArUco marker on the ground, centres itself above it, and lands—without any human input.

Software Stack

Component	Role
PX4 Autopilot	Stabilisation, flight modes, motor control
QGroundControl	Operator dashboard & telemetry monitor
Gazebo	3-D physics simulation, virtual camera
ROS 2 Humble	Inter-process message passing
MAVSDK (Python)	High-level drone command API
OpenCV + ArUco	Real-time visual marker detection

2 System Architecture

2.1 SITL Pipeline

Software-In-The-Loop (SITL) replaces real hardware with a virtual equivalent. The closed-loop pipeline is:

lightblue **Gazebo** (world + physics + camera)
↓ *sensor data*
lightblue **PX4 SITL** (autopilot in software)
↓ *MAVLink*
lightblue **ros_gz_bridge** → `/gimbal_camera`
↓ *Image topic*
lightblue **camera_node.py** (ArUco detection)
↓ `/aruco_error_x, /aruco_error_y`
lightblue **precision_landing.py** (state machine)
↓ *MAVSDK velocity commands*
lightblue **PX4 SITL** → **Gazebo** (loop)

2.2 Two-Script Design

The system is split into two independent ROS 2 nodes:

- `camera_node.py` — *perception*: receives images, detects the marker, publishes normalised errors.
- `precision_landing.py` — *control*: reads errors, runs a state machine, sends MAVSDK velocity commands.

Separating perception from control makes each script smaller, independently testable, and easy to extend.

3 Background Concepts

3.1 PX4 and QGroundControl

PX4 is open-source autopilot firmware that runs on Pixhawk flight-controller hardware. It handles low-level stabilisation, navigation, and exposes high-level flight modes:

- **RTL** — Return to Launch: GPS-guided flight back to takeoff point.
- **LAND** — Descend and land at current position.
- **OFFBOARD** — External computer streams velocity setpoints directly; used for precision landing.

QGroundControl (QGC) is the ground-control *application*—a laptop-side dashboard for telemetry monitoring, parameter editing, and mode switching. PX4 is the brain; QGC is the display.

3.2 Gazebo and ROS 2

Gazebo simulates the drone body, physics (gravity, drag, collisions), and sensors including the downward-facing gimbal camera whose pixel output is indistinguishable from a real camera.

ROS 2 is middleware: independent programs (*nodes*) exchange typed data over named channels (*topics*). One node *publishes*; others *subscribe*. The `ros_gz_bridge` package bridges Gazebo's internal transport to ROS 2 topics.

3.3 ArUco Markers and Normalised Error

An ArUco marker is a square binary fiducial tag. OpenCV's `cv2.aruco` module returns the four corner pixel coordinates of every visible marker. The marker centre (m_x, m_y) is the mean of those corners. Normalised error relative to image centre (c_x, c_y) is:

$$n_x = \frac{m_x - c_x}{c_x}, \qquad n_y = \frac{m_y - c_y}{c_y} \tag{1}$$

Both values lie in $[-1, 1]$; zero means the marker is perfectly centred. This formulation is resolution-independent.

3.4 Proportional Control

The controller uses a simple P-law to convert errors into body-frame velocity commands:

$$v_x = -K_p\, n_y, \qquad v_y = +K_p\, n_x, \tag{2}$$

$$v_z = 0 \text{ (ALIGN)},\ 0.2 \text{ m/s (DESCEND)},$$

$$\dot{\psi} = -1.2\, n_x \tag{3}$$

with $K_p = 0.6$. Signs follow the NED body frame: $+v_x$ is forward, $+v_y$ is right, $+v_z$ is *down*.

4 Script 1: `camera_node.py`

This node implements the perception pipeline shown in Fig. 1b.

4.1 Imports

```python
#!/usr/bin/env python3
# Tells the shell: run this with Python 3

import rclpy
# ROS2 client library. Entry point for all
# ROS2 operations (init, spin, shutdown).

from rclpy.node import Node
# Base class every ROS2 node must inherit.
# Provides create_subscription(), create_publisher(),
# get_logger(), etc.

from sensor_msgs.msg import Image
# Standard ROS2 image message. Carries raw pixel
# bytes, encoding string, and timestamp.

from std_msgs.msg import Float32
# Simple ROS2 message holding one 32-bit float.
# Used to publish nx and ny errors.

from cv_bridge import CvBridge
# Converts ROS2 Image <-> NumPy (OpenCV) array.
# Must match encoding (bgr8 here).

import cv2        # OpenCV: image processing + ArUco
import numpy as np  # NumPy: fast array math (mean)
```

4.2 Class and Constructor

```python
class ArucoDetector(Node):
# Inherits Node -> automatically gains all ROS2
# capabilities (publishers, subscribers, logging).

  def __init__(self):
    super().__init__('aruco_detector')
    # Registers this node in the ROS2 graph under
    # the name "aruco_detector".

    self.bridge = CvBridge()
    # One bridge instance reused every frame.
    # Creating it once avoids repeated allocation.

    # ---- SUBSCRIBER ----
    self.subscription = self.create_subscription(
        Image,            # expected message type
        '/gimbal_camera', # topic from ros_gz_bridge
        self.image_callback, # called on each frame
        10                # queue: buffer up to 10 msgs
    )
```

4.3 Publishers

```python
    # Horizontal error -> control script
    self.pub_x = self.create_publisher(
        Float32, '/aruco_error_x', 10)

    # Vertical error -> control script
    self.pub_y = self.create_publisher(
        Float32, '/aruco_error_y', 10)

    # Annotated camera frame for rqt_image_view
    self.debug_pub = self.create_publisher(
        Image, '/camera_debug', 10)
```

4.4 ArUco Detector Setup

```python
    # 4x4 binary grid, 50 unique marker IDs (0-49).
    # Fast detection; robust under varied lighting.
    self.aruco_dict = cv2.aruco.getPredefinedDictionary(
        cv2.aruco.DICT_4X4_50)

    # Detection thresholds (tunable).
    self.parameters = cv2.aruco.DetectorParameters()

    # Combines dictionary + parameters into one object.
    self.detector = cv2.aruco.ArucoDetector(
        self.aruco_dict, self.parameters)

    self.get_logger().info("Aruco detector started")
    # Logs to ROS2 console with timestamp.
```

4.5 Image Callback

```
def image_callback(self, msg):
  # ROS2 calls this automatically on every new frame.
  # 'msg' is a sensor_msgs/Image message.
  try:
    # Convert ROS2 Image to OpenCV BGR array.
    frame = self.bridge.imgmsg_to_cv2(
        msg, desired_encoding='bgr8')

    # Keep every 2nd pixel -> halve resolution.
    # 1280x720 becomes 640x360; 4x speedup.
    frame = frame[::2, ::2]

    H, W = frame.shape[:2]
    # shape = (height, width, channels).
    # [:2] drops the channel count.

    cx, cy = W // 2, H // 2
    # Integer centre coordinates.
    # All errors are measured from this point.

    debug_frame = frame.copy()
    # Draw on the copy; keep 'frame' clean for detection.

    # Cyan crosshair at image centre.
    cv2.line(debug_frame,(0,cy),(W,cy),(255,255,0),2)
    cv2.line(debug_frame,(cx,0),(cx,H),(255,255,0),2)

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    # ArUco detector requires single-channel input.

    corners, ids, _ = self.detector.detectMarkers(gray)
    # corners: list of 4 (x,y) pairs per marker.
    # ids:     marker ID array, or None if not found.
    # _:       rejected candidates (ignored).
```

4.6 Marker Found: Error Calculation

```
    if ids is not None:
      # At least one marker visible in this frame.

      cv2.aruco.drawDetectedMarkers(
          debug_frame, corners, ids)
      # Draws green bounding box + ID on debug frame.

      c = corners[0][0]
      # corners[0]: first detected marker.
      # [0] unwraps extra dimension added by OpenCV.
      # c is now shape (4, 2): four (x,y) corners.

      mx = int(np.mean(c[:, 0]))  # mean of x-coords
      my = int(np.mean(c[:, 1]))  # mean of y-coords
      # Geometric centre of the detected marker.

      # Normalised error (Eq. 1 in the paper):
      nx = (mx - cx) / cx   # in [-1, +1]
```

```
        ny = (my - cy) / cy   # in [-1, +1]

        # Publish Float32 messages.
        msgx = Float32(); msgx.data = float(nx)
        self.pub_x.publish(msgx)
        msgy = Float32(); msgy.data = float(ny)
        self.pub_y.publish(msgy)

        # Visual annotations on debug_frame:
        cv2.circle(debug_frame,(mx,my),5,(0,0,255),-1)
        # Solid red dot at marker centre.

        cv2.arrowedLine(debug_frame,(cx,cy),(mx,my),
                        (0,255,0),2)
        # Green arrow from image centre to marker.
        # Shows the error vector visually.

        cv2.putText(debug_frame,
            f"nx={nx:.2f}, ny={ny:.2f}",
            (10,40), cv2.FONT_HERSHEY_SIMPLEX,
            0.7, (0,255,0), 2)
        # Overlays numeric errors in top-left corner.
        # Visible in Fig. 1b.

      else:
        print("[ARUCO] Marker not detected")
        # No marker visible; control script receives
        # no new error updates this frame.

      cv2.imshow("Aruco Detection", debug_frame)
      cv2.waitKey(1)
      # waitKey(1): processes window events + refreshes.
      # Without this, imshow() shows nothing.

      # Convert back to ROS2 Image and publish.
      debug_msg = self.bridge.cv2_to_imgmsg(
          debug_frame, encoding='bgr8')
      self.debug_pub.publish(debug_msg)

    except Exception as e:
      self.get_logger().error(f"Camera error: {e}")
      # Node logs the error and continues; no crash.
```

4.7 Main Function

```
def main():
  rclpy.init()          # Start ROS2 runtime (once only)
  node = ArucoDetector() # Triggers __init__
  rclpy.spin(node)
  # Event loop: blocks here, calling image_callback()
  # every time a new frame arrives. Runs until Ctrl+C.
  node.destroy_node()   # Release ROS2 resources
  rclpy.shutdown()      # Stop ROS2 runtime
  cv2.destroyAllWindows() # Close OpenCV windows

if __name__ == "__main__":
  main()
```

5 Script 2: precision_landing.py

5.1 Imports

```python
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from std_msgs.msg import Float32
# (Same ROS2 imports as camera_node.)

import asyncio
# Python async runtime. MAVSDK is fully async:
# every drone command must be awaited inside an
# async function. Prevents blocking the process.

import threading
# Runs the MAVSDK async loop on a separate OS thread
# so it never blocks ROS2's spin loop.

import sys   # sys.stdout for in-place terminal output
import time  # time.time() for wall-clock timestamps

from mavsdk import System
# Main MAVSDK object. Represents the drone connection.
# All actions and telemetry go through this.

from mavsdk.offboard import VelocityBodyYawspeed
# OFFBOARD setpoint type:
#   (forward_m_s, right_m_s, down_m_s, yaw_deg_s)
```

5.2 Node Variables

```python
class PrecisionLanding(Node):
  def __init__(self):
    super().__init__('precision_landing')

    self.nx = 0.0   # Latest horizontal error from camera
    self.ny = 0.0   # Latest vertical error from camera
    # Initialised to 0 so drone holds position if no
    # camera message has arrived yet.

    self.last_detection_time = 0.0
    # Wall-clock time of the most recent received error.
    # Used to decide if the marker is "recently seen".

    self.first_detection_time = None
    # Wall-clock time the current detection streak began.
    # None = no marker seen yet (or streak was reset).

    self.state = "RTL"
    # State machine starts in RTL.
    # Transitions: RTL -> OFFBOARD_INIT -> ALIGN
    #              -> DESCEND -> (land + exit)

    self.current_z = 0.0
    # NED down_m: positive = below takeoff altitude.
    # Used to trigger final LAND when altitude < 0.5 m.
```

```

    # Subscribe to both error channels.
    self.create_subscription(
        Float32, '/aruco_error_x', self.x_cb, 10)
    self.create_subscription(
        Float32, '/aruco_error_y', self.y_cb, 10)

    # Create a fresh event loop for MAVSDK.
    self.loop = asyncio.new_event_loop()
    # Start MAVSDK in a daemon thread (auto-killed on
    # program exit).
    threading.Thread(
        target=self.start_loop, daemon=True).start()
```

5.3 Subscriber Callbacks

```
def x_cb(self, msg):
  self.nx = msg.data          # Update shared variable
  self.last_detection_time = time.time()
  # Record that the marker was visible right now.
  if self.first_detection_time is None:
    self.first_detection_time = time.time()
    # Start the stability clock on first detection.

def y_cb(self, msg):
  self.ny = msg.data
  self.last_detection_time = time.time()
  if self.first_detection_time is None:
    self.first_detection_time = time.time()
```

5.4 MAVSDK Thread and Connection

```
def start_loop(self):
  asyncio.set_event_loop(self.loop)
  # Assign self.loop to this thread (required when
  # creating a loop manually).
  self.loop.run_until_complete(self.run())
  # Blocks this thread until run() finishes (landing).

async def run(self):
  drone = System()
  await drone.connect(system_address="udp://:14540")
  # UDP port 14540 is PX4 SITL's default MAVLink port.

  async for state in drone.core.connection_state():
    if state.is_connected:
      print("Connected"); break
  # Async generator streams connection events.
  # break exits after confirming connection.

  await drone.action.return_to_launch()
  # Commands PX4 to enter RTL mode.
  # PX4 handles all navigation; we just monitor.

  Kp = 0.6   # Proportional gain (see Eq. 2)
```

5.5 State Machine

```
    while True:
      # Read altitude once per loop iteration.
      async for pos in drone.telemetry.position_velocity_ned():
        self.current_z = pos.position.down_m; break
      # NED down_m: negative = above takeoff point.

      now = time.time()

      # --- Detection timing logic ---
      detected_recent = (now - self.last_detection_time) < 0.5
      # True if marker was seen within the last 0.5 s.

      detected_stable = (
          self.first_detection_time is not None and
          (now - self.first_detection_time) > 3.0
      )
      # True if marker has been continuously visible
      # for at least 3 seconds without interruption.

      if not detected_recent:
        self.first_detection_time = None
        # Marker disappeared -> reset stability clock.
```

5.5.1 State 1: RTL

```
      if self.state == "RTL":
        sys.stdout.write("\r[RTL] Returning home...")
        sys.stdout.flush()
        # \r overwrites current line (no scroll).
        # flush() forces immediate display.

        if detected_stable:
          print("\nMarker stable for 3s -> OFFBOARD_INIT")
          self.state = "OFFBOARD_INIT"
          # Marker confirmed for 3 s -> safe to land.
```

5.5.2 State 2: OFFBOARD_INIT

```
      elif self.state == "OFFBOARD_INIT":
        for _ in range(50):
          await drone.offboard.set_velocity_body(
              VelocityBodyYawspeed(0, 0, 0, 0))
          await asyncio.sleep(0.05)
        # PX4 requirement: stream setpoints for >= 2 s
        # before offboard.start() or it is rejected.
        # 50 x 0.05 s = 2.5 s of pre-streaming.

        try:
          await drone.offboard.start()
          print("OFFBOARD started")
          self.state = "ALIGN"
        except Exception as e:
          print(f"OFFBOARD failed: {e}"); return
```

5.5.3 State 3: ALIGN

```
      elif self.state == "ALIGN":
        vx = -Kp * self.ny  # forward/back correction
        vy = +Kp * self.nx  # left/right correction
        vz = 0.0            # no descent during align
        yaw_rate = -1.2 * self.nx
        # Rotates nose toward marker.
        # nx>0 (marker right) -> negative yaw -> CW.

        await drone.offboard.set_velocity_body(
            VelocityBodyYawspeed(vx, vy, vz, yaw_rate))

        if abs(self.nx) < 0.1 and abs(self.ny) < 0.1:
          print("\nAligned -> DESCEND")
          self.state = "DESCEND"
          # Both errors < 10% of half-image -> centred.
```

5.5.4 State 4: DESCEND

```
      elif self.state == "DESCEND":
        vx = -Kp * self.ny  # continue correction
        vy = +Kp * self.nx
        vz = 0.2            # 0.2 m/s downward (NED+)
        yaw_rate = -1.0 * self.nx

        await drone.offboard.set_velocity_body(
            VelocityBodyYawspeed(vx, vy, vz, yaw_rate))

        if abs(self.current_z) < 0.5:
          print("\nAltitude < 0.5 m -> LAND")
          await drone.action.land()
          break
          # PX4 LAND mode handles final touchdown.
          # break exits while True; run() returns.

      await asyncio.sleep(0.05)  # 20 Hz loop rate
```

5.6 Main Function

```
def main():
  rclpy.init()
  node = PrecisionLanding()
  # __init__ creates subscribers AND starts MAVSDK
  # thread immediately.

  try:
    rclpy.spin(node)
    # Processes ROS2 callbacks (x_cb, y_cb) forever.
  except KeyboardInterrupt:
    pass  # Ctrl+C exits cleanly
  finally:
    node.destroy_node(); rclpy.shutdown()
    # Always runs: releases ROS2 resources.

if __name__ == '__main__':
  main()
```

6 State Machine Summary

State	Entry condition	Exit condition
RTL	Program start	Marker stable $\geq 3\,\mathrm{s}$
OFFBOARD_INIT	After RTL	50 setpoints streamed
ALIGN	After OFFBOARD	$\lvert n_x\rvert, \lvert n_y\rvert < 0.1$
DESCEND	After ALIGN	Altitude $< 0.5\,\mathrm{m}$
LAND	After DESCEND	PX4 touchdown

7 Setup & Running

7.1 Prerequisites (Ubuntu 22.04)

```
# 1. PX4 SITL
git clone https://github.com/PX4/PX4-Autopilot.git --recursive
bash PX4-Autopilot/Tools/setup/ubuntu.sh

#2 .
# Add Gazebo repository
sudo wget https://packages.osrfoundation.org/gazebo.gpg -O /usr/share/keyrings/
   pkgs-osrf-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/pkgs-
   osrf-archive-keyring.gpg] http://packages.osrfoundation.org/gazebo/ubuntu-
   stable $(lsb_release -cs) main" | sudo tee /etc/apt/sources.list.d/gazebo-
   stable.list > /dev/null

# Update and install Gazebo
sudo apt update
sudo apt install -y gz-garden

# Verify installation
gz --version

# 3. ROS2 Humble
sudo apt install ros-humble-desktop
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc

# 4. Bridge + Python deps
sudo apt install ros-humble-ros-gz ros-humble-cv-bridge
pip install mavsdk opencv-python numpy
```

7.2 Launch Order (4 terminals)

```
# T1: PX4 SITL + Gazebo
cd PX4-Autopilot && make px4_sitl gz_x500_gimbal

# T2: ROS-Gazebo bridge
ros2 run ros_gz_bridge parameter_bridge \
  /gimbal_camera@sensor_msgs/msg/Image@gz.msgs.Image

# T3: Vision node
python3 camera_node.py

# T4: Precision landing node
python3 precision_landing.py
```

8 Troubleshooting

Issue	Fix
OFFBOARD rejected	Pre-stream setpoints $\geq 2\,\mathrm{s}$ before `offboard.start()`.
Marker not found	Check topic is publishing; improve lighting; increase print size.
Drone drifts	Reduce K_p (try 0.3); check marker is centred at startup.
MAVSDK refused	Start PX4 SITL first; default port `udp://:14540`.
Topic not found	Verify `ros_gz_bridge` is running; match topic name exactly.
Overshoot on land	Reduce `vz` (try $0.1\,\mathrm{m/s}$) or tighten altitude threshold.

9 Conclusion

This document presented a complete, beginner-accessible guide to an autonomous precision landing system for multirotors. The two-node architecture cleanly separates perception (`camera_node.py`) from control (`precision_landing.py`), making each component independently testable. The proportional controller (2) drives both normalised errors to zero before descent begins, and PX4's LAND mode handles the final touchdown.

The system runs unchanged on real hardware by replacing the SITL `udp://:14540` address with the physical flight-controller connection string (e.g. `serial:///dev/ttyUSB0:57600`).

Tip: Always validate every code change in SITL simulation before deploying to a real drone. The ArUco detection threshold, K_p, and descent rate should be tuned in simulation first.

Further reading: PX4 documentation at https://docs.px4.io/main/en/ covers assembly, sensor calibration, and all flight modes in detail.